

Class 7: Machine Learning 1

Mason Dinh (PID: A19140455)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	9
Principal Component Analysis (PCA)	11
Analysis of UK food data	11
Data Import	11
Tidy data	17
Exploratory analysis	18
PCA to the rescue	19

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we can use `rnorm()` function:

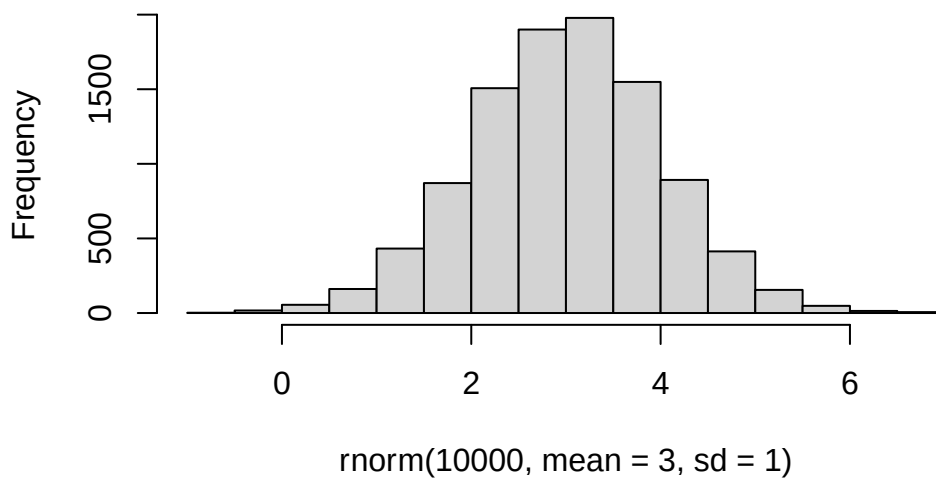
```
rnorm(100)
```

```
[1] -0.65264098 -0.26066566  0.61970519  0.22107028  1.66866927 -0.46813820
[7]  0.02368421  0.19766271 -0.24393098  0.79058856  0.56792848  0.02138457
[13] -0.43741235  0.55078709 -2.71559450  0.06534629 -0.49324238 -0.37492216
[19]  0.30752161  1.27044891  1.74348382  0.03456189 -0.49739296  1.11647024
```

```
[25] 0.92785406 0.81529166 0.72489090 -0.63941579 0.64229915 0.78750724
[31] 0.19504807 0.62829710 -2.24844916 -1.07534018 -0.65547323 1.54351915
[37] 0.06510287 1.15679001 -0.70941710 -2.76942877 0.91685320 -1.10705424
[43] 0.40830295 0.47625245 0.15810645 1.44988773 0.84612731 1.25365800
[49] 1.67456615 -0.86347897 -2.82544406 -1.57629220 -0.62444419 -0.33528461
[55] -1.00955731 0.39982376 -0.29253200 1.34884007 -1.24191124 1.56589750
[61] 0.16515676 -1.38371765 0.80948723 0.05677326 -0.08490313 1.04184141
[67] 0.59387804 0.78498663 0.21546571 -0.47349396 -0.59080088 0.64047569
[73] -1.96471967 0.17773272 -0.13717710 1.21656528 1.47323157 1.22885665
[79] -0.62571543 1.36097418 1.33787457 -1.05609789 -1.66954732 0.59997203
[85] -0.12445421 -0.65011507 0.03566069 0.19862545 -1.07813755 -0.82545339
[91] 0.16072266 0.61582814 0.39460197 -1.58868859 -0.72997678 -0.49343849
[97] 0.04361828 -0.46345047 -0.37861621 0.49169156
```

```
hist(rnorm(10000,mean = 3, sd=1))
```

Histogram of rnorm(10000, mean = 3, sd = 1)



```
x<- c(rnorm(30,-3), rnorm(30,+3))
x
```

```
[1] -4.734266 -3.117291 -2.582545 -2.291784 -4.191444 -1.980645 -3.422001
[8] -4.482982 -2.573216 -3.706123 -2.796997 -2.710503 -3.647318 -3.124342
[15] -3.339479 -2.823803 -2.481633 -2.154909 -1.108659 -2.147058 -2.885118
```

```
[22] -3.093344 -1.264742 -1.581126 -4.719911 -2.478703 -5.231880 -2.448784
[29] -2.594548 -2.387789  2.104783  3.408281  1.444275  2.304270  2.923106
[36]  3.216730  3.371286  3.586490  5.706420  1.062296  2.170459  3.259546
[43]  4.057196  4.253745  2.400947  2.848077  3.343632  4.707340  2.171934
[50]  2.369512  3.378257  3.262299  1.845055  1.079647  2.200900  2.624226
[57]  4.153965  2.123599  3.993282  3.156324
```

```
rev(x)
```

```
[1]  3.156324  3.993282  2.123599  4.153965  2.624226  2.200900  1.079647
[8]  1.845055  3.262299  3.378257  2.369512  2.171934  4.707340  3.343632
[15]  2.848077  2.400947  4.253745  4.057196  3.259546  2.170459  1.062296
[22]  5.706420  3.586490  3.371286  3.216730  2.923106  2.304270  1.444275
[29]  3.408281  2.104783 -2.387789 -2.594548 -2.448784 -5.231880 -2.478703
[36] -4.719911 -1.581126 -1.264742 -3.093344 -2.885118 -2.147058 -1.108659
[43] -2.154909 -2.481633 -2.823803 -3.339479 -3.124342 -3.647318 -2.710503
[50] -2.796997 -3.706123 -2.573216 -4.482982 -3.422001 -1.980645 -4.191444
[57] -2.291784 -2.582545 -3.117291 -4.734266
```

```
z<-cbind(x = x, y = rev(x))
z
```

	x	y
[1,]	-4.734266	3.156324
[2,]	-3.117291	3.993282
[3,]	-2.582545	2.123599
[4,]	-2.291784	4.153965
[5,]	-4.191444	2.624226
[6,]	-1.980645	2.200900
[7,]	-3.422001	1.079647
[8,]	-4.482982	1.845055
[9,]	-2.573216	3.262299
[10,]	-3.706123	3.378257
[11,]	-2.796997	2.369512
[12,]	-2.710503	2.171934
[13,]	-3.647318	4.707340
[14,]	-3.124342	3.343632
[15,]	-3.339479	2.848077
[16,]	-2.823803	2.400947
[17,]	-2.481633	4.253745
[18,]	-2.154909	4.057196

[19,]	-1.108659	3.259546
[20,]	-2.147058	2.170459
[21,]	-2.885118	1.062296
[22,]	-3.093344	5.706420
[23,]	-1.264742	3.586490
[24,]	-1.581126	3.371286
[25,]	-4.719911	3.216730
[26,]	-2.478703	2.923106
[27,]	-5.231880	2.304270
[28,]	-2.448784	1.444275
[29,]	-2.594548	3.408281
[30,]	-2.387789	2.104783
[31,]	2.104783	-2.387789
[32,]	3.408281	-2.594548
[33,]	1.444275	-2.448784
[34,]	2.304270	-5.231880
[35,]	2.923106	-2.478703
[36,]	3.216730	-4.719911
[37,]	3.371286	-1.581126
[38,]	3.586490	-1.264742
[39,]	5.706420	-3.093344
[40,]	1.062296	-2.885118
[41,]	2.170459	-2.147058
[42,]	3.259546	-1.108659
[43,]	4.057196	-2.154909
[44,]	4.253745	-2.481633
[45,]	2.400947	-2.823803
[46,]	2.848077	-3.339479
[47,]	3.343632	-3.124342
[48,]	4.707340	-3.647318
[49,]	2.171934	-2.710503
[50,]	2.369512	-2.796997
[51,]	3.378257	-3.706123
[52,]	3.262299	-2.573216
[53,]	1.845055	-4.482982
[54,]	1.079647	-3.422001
[55,]	2.200900	-1.980645
[56,]	2.624226	-4.191444
[57,]	4.153965	-2.291784
[58,]	2.123599	-2.582545
[59,]	3.993282	-3.117291
[60,]	3.156324	-4.734266

```
p <- 1:5  
cbind(p, rev(p))
```

```
p
[1,] 1 5
[2,] 2 4
[3,] 3 3
[4,] 4 2
[5,] 5 1
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km<-kmeans(z, centers = 2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-2.936765	2.950929
2	2.950929	-2.936765

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 61.66942 61.66942
(between_SS / total_SS = 89.4 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
$class  
[1] "kmeans"
```

Q. how many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What “components of your result object details cluster assignment/membership?

```
km$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2  
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

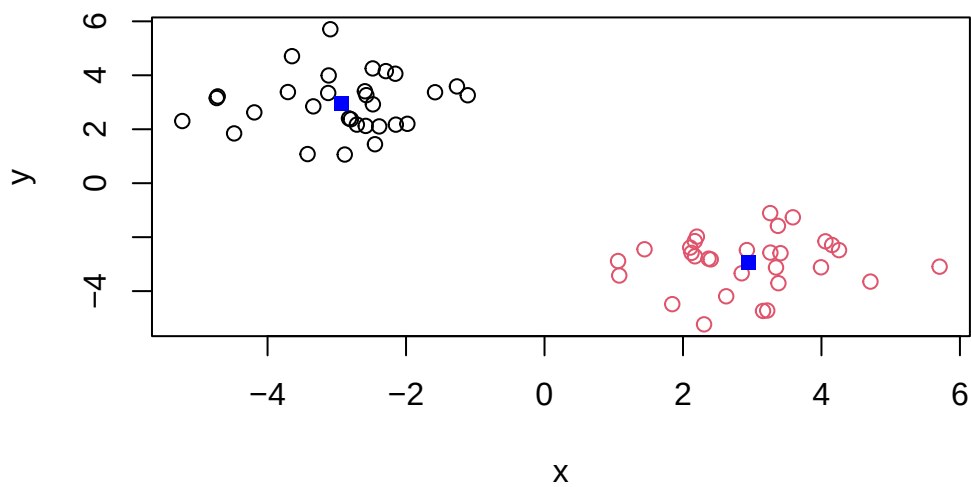
Q. What “components of your result object details cluster center?

```
km$centers
```

```
      x      y  
1 -2.936765  2.950929  
2  2.950929 -2.936765
```

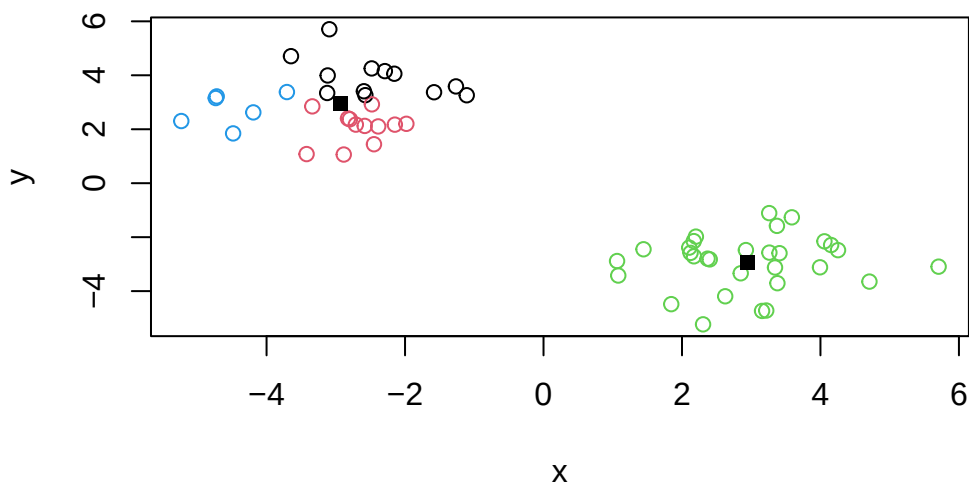
Q. plot z colored by the kmeans cluster assignment and add cluster centers as blue points.

```
plot(z,col=km$cluster)  
points(km$centers,col="blue",pch=15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)?

```
km4 <- kmeans(z, centers =4 )  
plot(z, col = km4$cluster)  
points(km$centers,col="black",pch=15)
```



N.B. you need to tell K-means the number of clusters (i.e. set `centers=2`)!!

```
km$totss
```

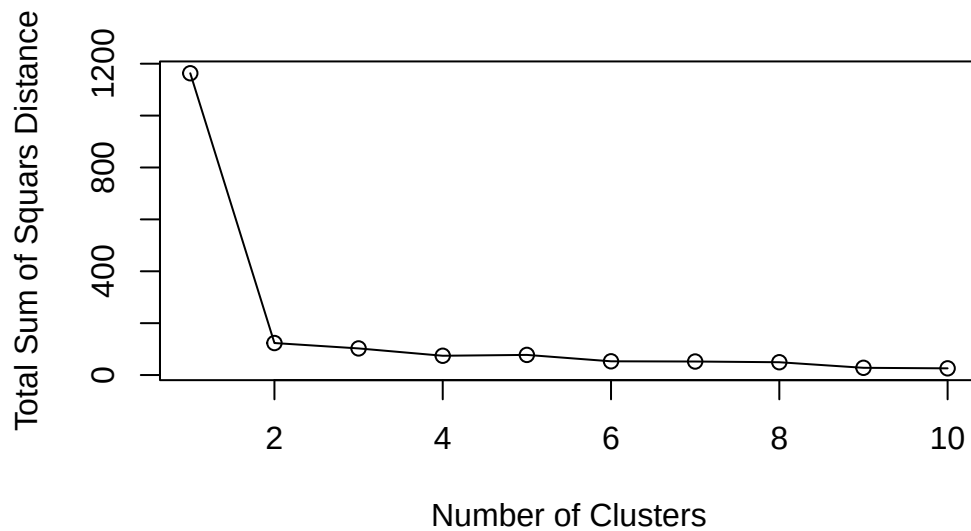
```
[1] 1163.287
```

One approach is to try different values for `centers` and then pick the best...

```
ans<- NULL
for(i in 1:10) {
  km <- kmeans(z,centers=i)
  ans<-c(ans,km$tot.withinss)
}
ans
```

```
[1] 1163.28710 123.33884 102.58263 74.33414 77.60195 52.97769
[7] 52.17915 49.34101 27.90689 25.79873
```

```
plot(ans, typ="o", xlab= "Number of Clusters", ylab="Total Sum of Squares Distance")
```

Hierarchical Clustering

The main function in “base” R for hierarchical clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

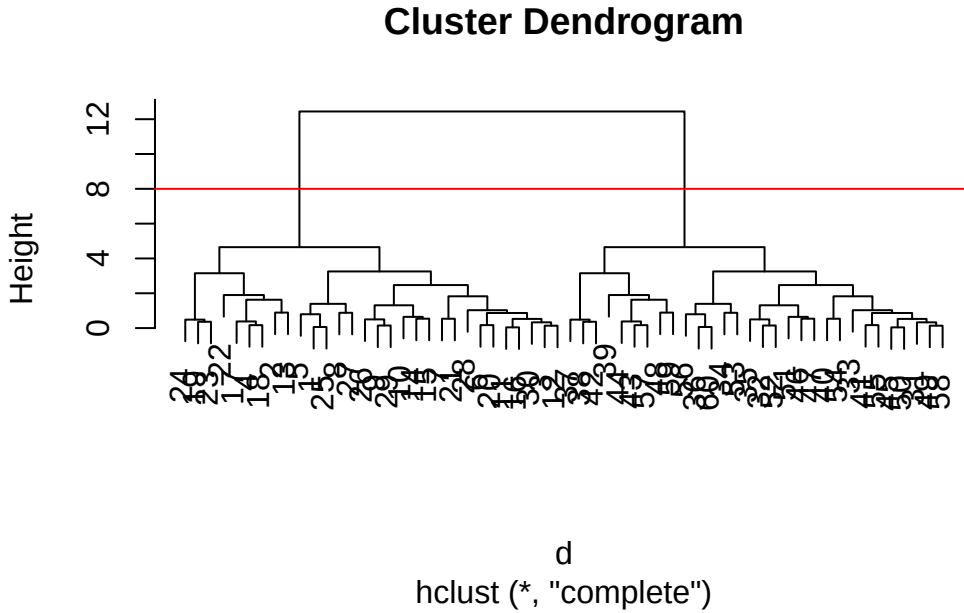
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8,col="red")
```



Once we have our `hclust()` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc,h=8)
```

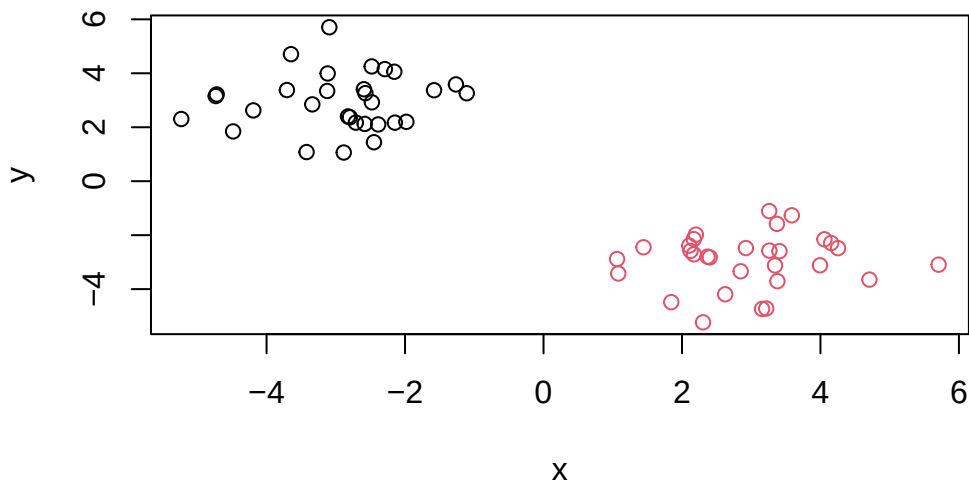
[illegible]

```
cutree(hc, k=4)
```

[1] 1 2 1 2 1 1 1 1 1 1 1 2 1 1 1 2 2 2 1 1 2 2 2 1 1 1 1 1 3 3 3 3 3 3 4 4
[39] 4 3 3 4 4 4 3 3 3 4 3 3 3 3 3 3 3 4 3 4 3

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
grps<-cutree(hc,k=2)
plot(z, col=grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66

2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17 5
```

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267

Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17  4
```

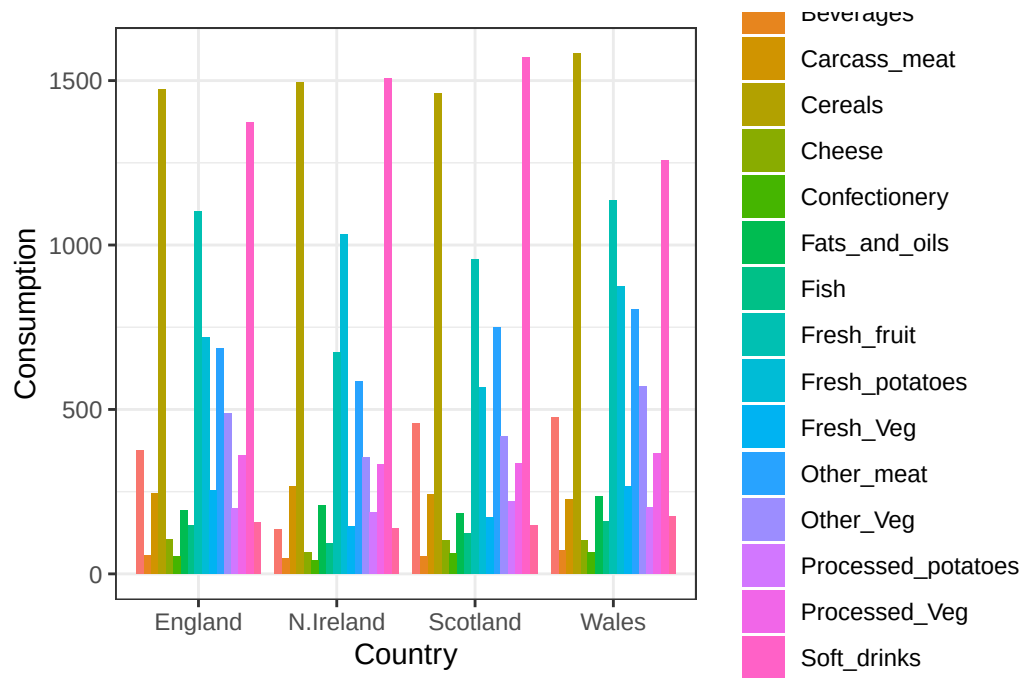
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second method to solving the “row-names problem” because it is much more convenient and can be done much faster than the first method. However, I think the first method is much more robust because it gives the user a lot more control over the output of the code.

```
library(tidyr)
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")
dim(x_long)
```

```
[1] 68  3
```

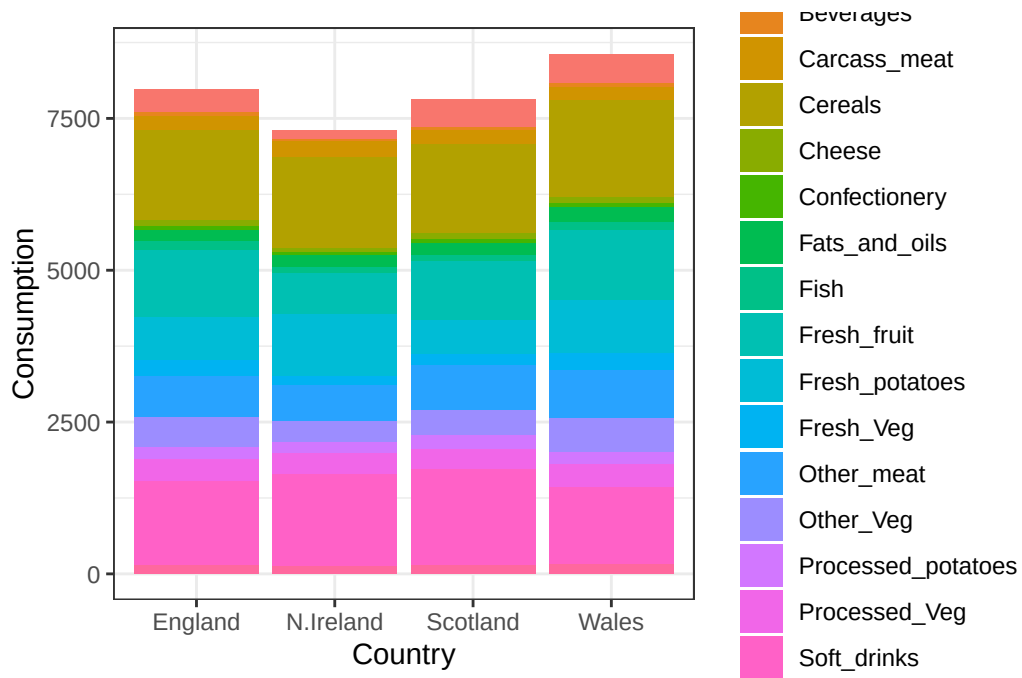
```
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

Changing the position from “dodge” to “stack” will result in a stacked barplot.

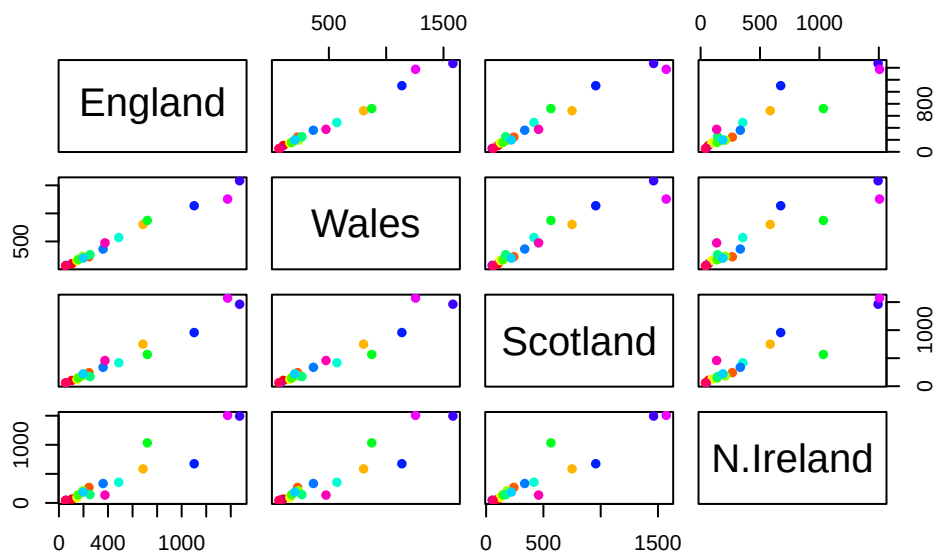
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

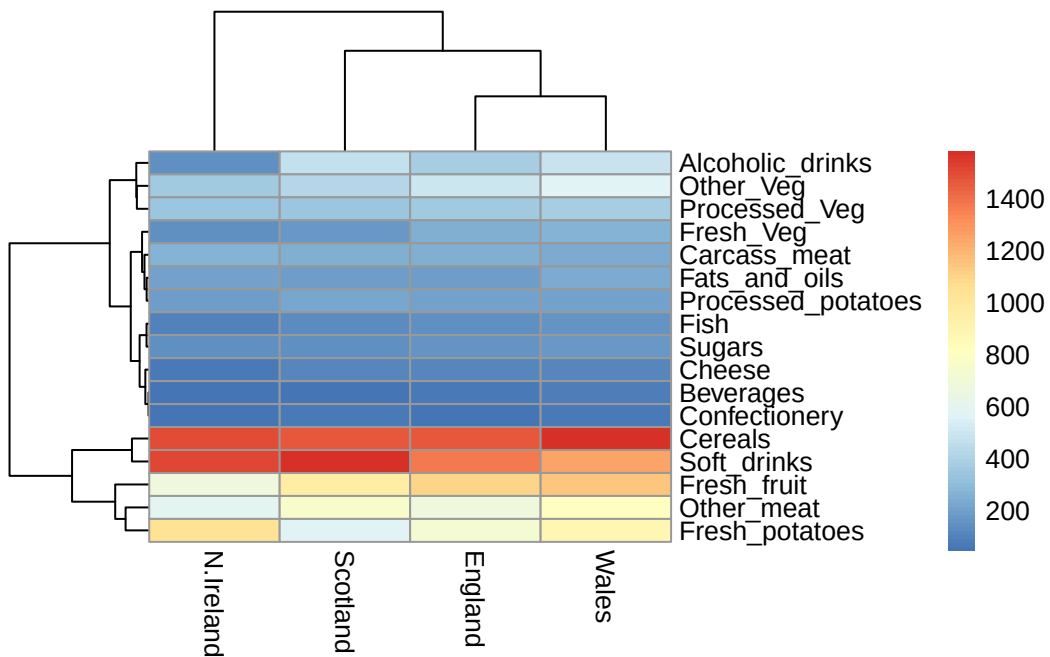
The `pairs()` function creates scatterplots that show the relationship between every pair of variables in the dataset. If a point lies on a diagonal line in the figure, it means the two variables that are being compared have the same value.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

The countries that cluster together are England, Wales, and Scotland because from the plots we can see that their points follow similar trends. This suggest that these three countries share similar food consumption patterns. You can tell that there is a difference between Northern Ireland and the other countries but its hard to see anything more specific.

Tidy data

```
library(tidyr)
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")
dim(x_long)
```

```
[1] 68  3
```

```
head(x_long)
```

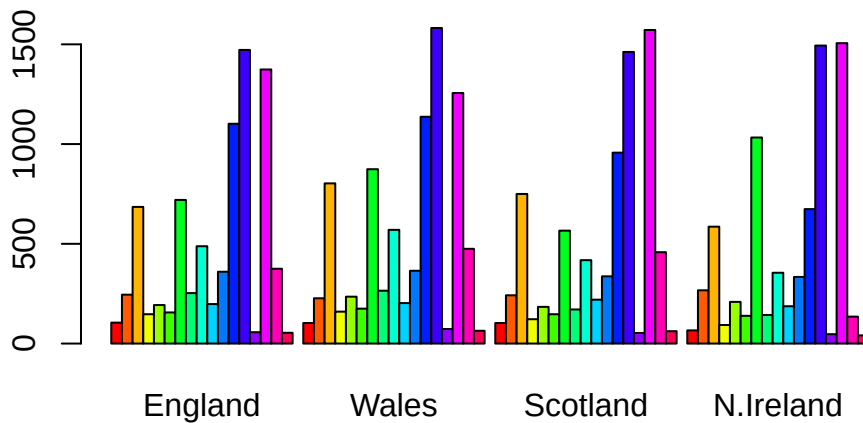
```
# A tibble: 6 x 3
  Food          Country Consumption
<chr>         <chr>         <int>
1 "Cheese"      England          105
2 "Cheese"      Wales            103
3 "Cheese"      Scotland         103
4 "Cheese"      N.Ireland         66
5 "Carcass_meat " England          245
6 "Carcass_meat " Wales            227
```

Fix anything that went wrong with data import.

Exploratory analysis

Make some plots to help make sense of obvious trends...

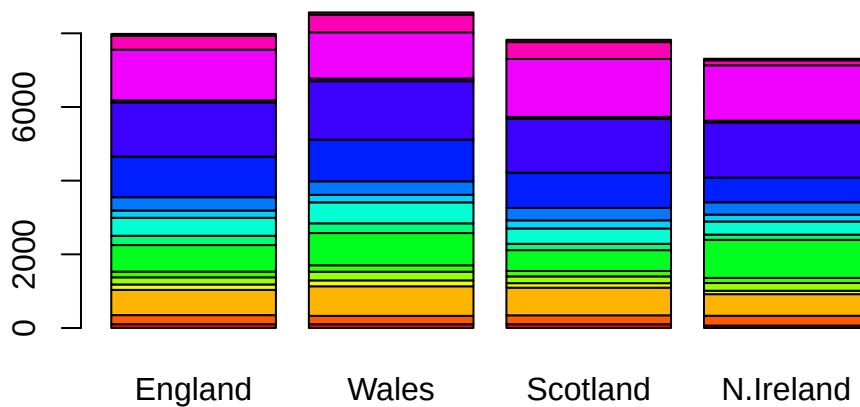
```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

Changing the `beside()` function from `TRUE` to `FALSE` will stack the bars.

```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



Key-point: Even relatively small datasets can prove challenging to interpret

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object

```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our main result figures:

```
attributes(pca)
```

```
$names
```

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

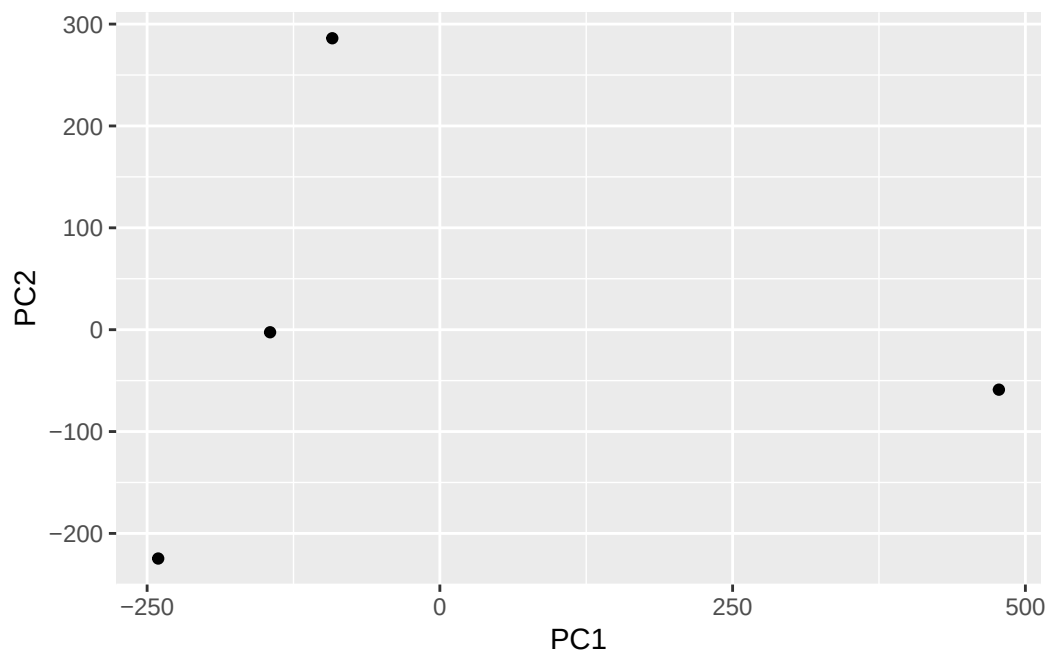
```
$class
```

```
[1] "prcomp"
```

The main result figure from this analysis is called a “PC score plot” or “ordination plot” or “PC plot” or PC1 vs PC2 plot.

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

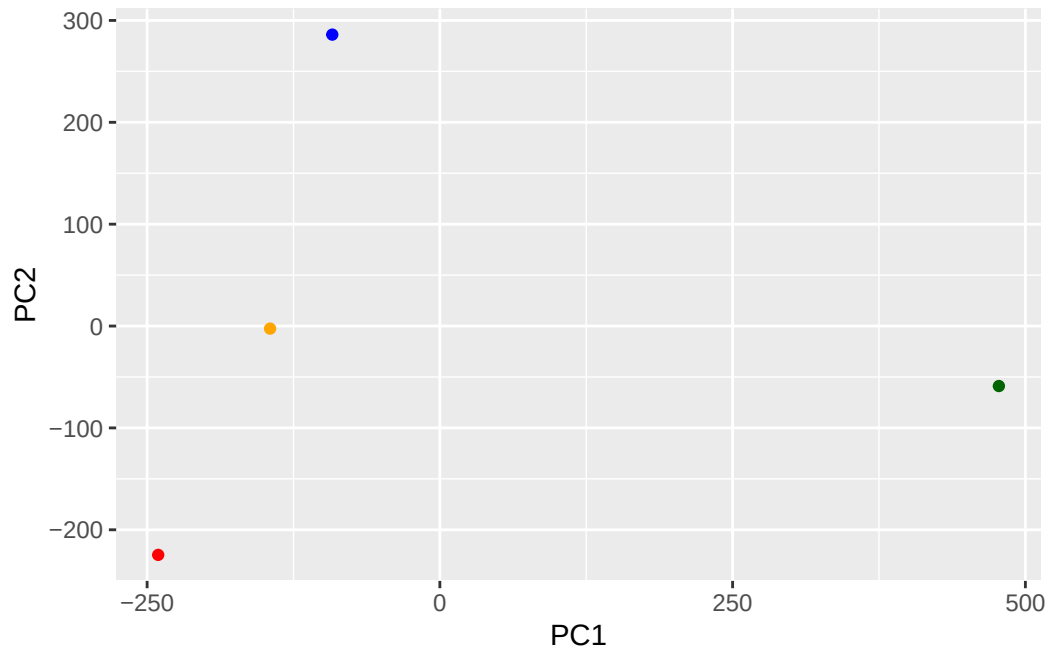
```
library(ggplot2)
ggplot(pca$x) + aes(PC1,PC2) + geom_point()
```



```
mycols<-c("orange","red","blue","darkgreen")
```

```
library(ggplot2)
```

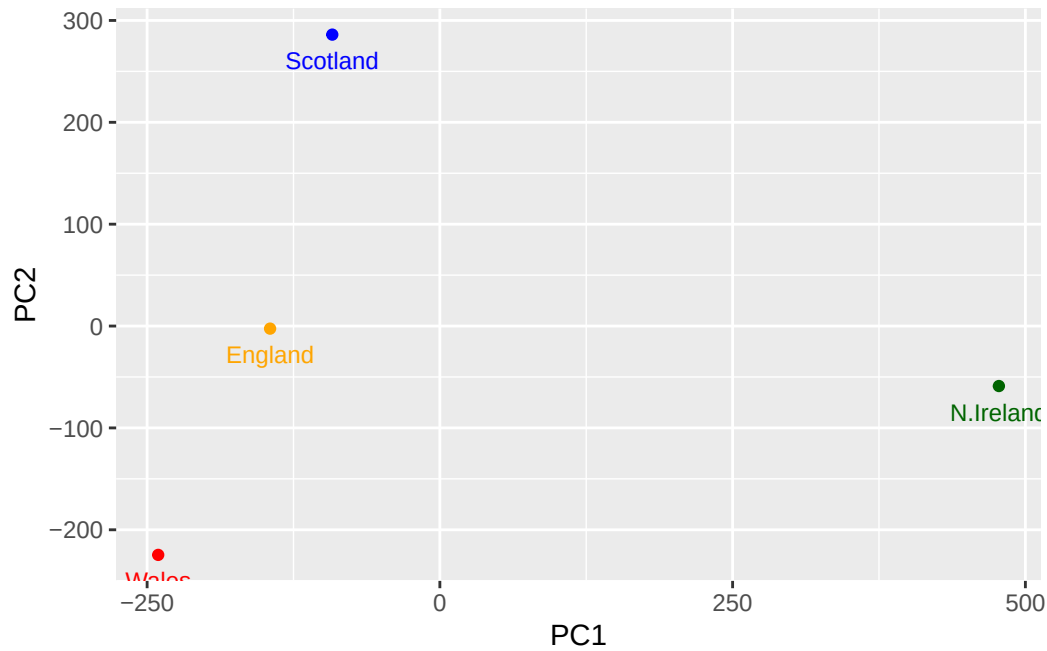
```
ggplot(pca$x) + aes(PC1,PC2) + geom_point(col=mycols)
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
mycols<-c("orange","red","blue","darkgreen")

library(ggplot2)
ggplot(pca$x) + aes(PC1,PC2,label=row.names(pca$x)) + geom_point(col=mycols) + geom_text(siz
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

The two food groups are soft drinks and fresh potatoes. PC2 mainly tells us about the variables that distinguish certain regions. PC2 tells us about the contrast between the food type and the country.

```
ggplot(pca$rotation)+ aes(PC1, row.names(pca$rotation)) + geom_col()
```

